Calcolo Parallelo
“*Cheat Sheet*”

Moreno Marzolla
Ultimo aggiornamento 2026-04-24

Performance models

$T_p(n)$ = Wall-clock time di un programma parallelo usando p unità di esecuzione su un input di dimensione n.

Work = $T_1(n)$

Span = $T_p(n)$

Cost = $p\,T_p(n)$

Overhead = $p\,T_p(n) - T_1(n)$

$p\,T_p(n) \geq T_1(n)$ (*Work law*)

$T_p(n) \geq T_\infty(n)$ (*Span law*)

$S(p) = T_1(n) / T_p(n)$ (*Speedup*)

$E(p) = S(p) / p$ (*Strong scaling efficiency*)

Principali direttive OpenMP

```
#pragma omp parallel
```

Parallel region, teams of threads, structured block, interleaved execution across threads.

```
int omp_get_thread_num()
int omp_get_num_threads()
omp_get_max_threads()
```

Get the ID of the current thread within the team, the size of the currently active team, and the maximum (default) size of the team.

```
double omp_get_wtime()
```

Timing blocks of code.

```
setenv OMP_NUM_THREADS N
export OMP_NUM_THREADS=N
```

Set the default number of threads with an environment variable.

```
#pragma omp barrier
#pragma omp critical
#pragma omp atomic
```

Synchronization, critical sections.

```
#pragma omp for
#pragma omp parallel for
```

Worksharing, parallel loops.

```
reduction(op:var)
```

Reduction of values across a team of threads.

```
schedule(dynamic [,chunksize])
schedule(static [,chunksize])
```

Loop schedules.

```
private(list)
shared(list)
firstprivate(list)
```

Data environment.

```
collapse(k)
```

Collapse k nested loops.

```
#pragma omp flush(list)
```

Ensure memory consistency.

```
#pragma omp master
#pragma omp single
```

Worksharing with a single thread; implicit barrier at end of "single",.

```
#pragma omp task
#pragma omp taskwait
```

Tasking construct, may include the data environment specification using shared(), firstprivate(), etc.

Principali funzioni MPI

```
int MPI_Init(argc, argv)
int MPI_Finalize()
```

Initialization and termination of the MPI program.

```
int MPI_Abort(comm, errorcode)
```

Abort the computation.

```
int MPI_Comm_rank(comm, rank)
int MPI_Comm_size(comm, size)
```

Timing blocks of code.

```
int MPI_Send(buf, count, datatype,
dest, tag, comm)
int MPI_Recv(buf, count, datatype,
source, tag, comm, status)
```

Point-to-point communication.

```
MPI_Barrier(comm)
```

Barrier synchronization.

```
MPI_Bcast(buf, count, datatype, root,
comm)
```

Broadcast.

```
int MPI_Scatter(sendbuf, sendcount,
sendtype, recvbuf, recvcount,
recvtype, root, comm)
int MPI_Gather(sendbuf, sendcount,
sendtype, recvbuf, recvcount,
recvtype, root, comm)
MPI_Allgather(sendbuf, sendcount,
sendtype, recvbuf, recvcount,
recvtype, comm)
```

Data distribution and gathering.

```
int MPI_Reduce(sendbuf, recvbuf,
count, datatype, op, root, comm)
MPI_Allreduce(sendbuf, recvbuf, count,
datatype, op, comm)
```

Reduction.

```
MPI_Scan(sendbuf, recvbuf, count,
datatype, op, comm)
```

Scan (e.g., prefix sum).

```
MPI_Alltoall(sendbuf, sendcount,
sendtype, recvbuf, recvcnt, recvtype,
comm)
```

Broadcast/transpose.

Principali costrutti CUDA

```
cudaMalloc(pointer, size)
cudaFree(pointer)
```

Memory management.

```
cudaMemcpy(dst, src, size, direction)
```

Memory copy.

```
__global__, __device__, __host__
```

Code/data target qualifier.

```
__shared__
```

Shared memory qualifier.

```
__syncthreads__
```

Intra-block thread synchronization.

```
func<<<BLOCKS, THREADS>>>(param)
```

Kernel launch.

```
cudaDeviceSynchronize()
```

Wait for completion of pending operations on the device.

Master Theorem

L'equazione di ricorrenza

$$T(n)=\begin{cases} aT(n/b)+f(n) & \text{se } n>1 \\ 1 & \text{se } n=1 \end{cases}$$

ha soluzione:

1. $T(n)=\Theta(n^{\log_b a})$ se $f(n)=o(n^{\log_b a})$;
2. $T(n)=\Theta(n^{\log_b a}\log n)$ se $f(n)=\Theta(n^{\log_b a})$;
3. $T(n)=\Theta(f(n))$ se $f(n)=\omega(n^{\log_b a})$ e $af(n/b) < cf(n)$ per qualche $c<1$ ed n sufficientemente grande.

Si ricorda che o($g(n)$) (“o piccolo”) è l'insieme delle funzioni il cui ordine di crescita è strettamente minore di quello di $g(n)$, Θ($g(n)$) (“Theta maiuscolo”) l'insieme di funzioni il cui ordine di crescita è uguale a quello di $g(n)$, e ω($g(n)$) (“omega minuscolo”) l'insieme di funzioni il cui ordine di crescita è strettamente maggiore di quello di $g(n)$.

Sommatorie notevoli

$$1+2+3+\ldots+n = \Theta(n^2)$$

$$1+a+a^2+\ldots+a^n = \frac{a^{n+1}-1}{a-1}$$

$$1+\frac{1}{2}+\frac{1}{4}+\ldots = O(1)$$